

Strategic Prototyping for Developing Big Data Systems

Hong-Mei Chen and Rick Kazman, University of Hawaii at Manoa

Serge Haziyeu, SoftServe

// A proposed model provides cost-effective, systematic risk management in agile big data system development. It uses prototyping strategically and only in areas that architecture analysis can't sufficiently address. Developers use less costly prototypes instead of blindly building full-scale prototypes. //



BIG DATA SYSTEM development faces more and greater risks than traditional small-data system development. These risks stem from^{1,2}

- the five *v*'s of big data (volume, variety, velocity, veracity, and value),
- paradigm shifts,
- rapid proliferation of big data technology,
- rapid technology changes,
- the difficulty of selecting big data technology,
- the complex integration of new and old systems, and
- the short history of big data system development.

For example, small-data systems focus on data at rest (stored in the database). In contrast, big data

systems emphasize data in use and data in motion, which drives data management to unfamiliar territory such as NoSQL.

Moreover, in small-data system development, data-program independence is a core principle. Architecture decisions are relatively trivial. In big data system development, data-program dependence is the norm, so architecture design becomes critical and poses great risks if done poorly.² Many new architectural patterns have recently become dominant in big data development, such as horizontal scaling, data lakes, polyglot persistence, the Lambda Architecture, and massively parallel processing. And the system development paradigm has shifted, from focusing on coding in small-data system development to orchestration and technology selection in big data systems. There are a large number of emerging big data technologies (both open source and proprietary), many of which are provided by new vendors or open source projects. This heightens risks in technology selection.

So, cost, schedule, and quality risks are more pronounced in big data system development. Managing—identifying, understanding, and mitigating—these risks is critical for system success, and the methods for doing this differ for big data system development.

To provide cost-effective, systematic risk management in agile big data system development, we developed the RASP (Risk-Based, Architecture-Centric Strategic Prototyping) model. An embedded multiple-case study of nine big data projects at SoftServe, a global outsourcing firm, validated RASP. On the basis of our experiences, we developed techniques to help architects deal with strategic prototyping.



Toward Strategic Prototyping

Traditionally, small-data management, following the rapid-application-development methodology, primarily and implicitly relies on *horizontal evolutionary prototyping* to help manage risks. Evolutionary prototyping is needed when requirements are uncertain, technologies are new, no comparable system has been previously developed, or experimentation or design evaluation is necessary to assess solutions. In horizontal evolutionary prototyping, all system functions are developed with limited, but increasing functionality per release.

However, this approach is challenged by the data scale and complexity of big data systems. For example, our early studies (not yet published) revealed that data retrieval performance with a 100-node Hadoop system differed dramatically from that with a 500-node system. It's naive (and incorrect) to assume linear scale-up. The scarcity of benchmarks in big data systems aggravates these problems. Creating prototypes has always been expensive and time-consuming and is even more so in big data system development. And prototyping can't identify many risks, such as vendor or product survivability for commercial products, and the community's size and vigor for open source products.

Alternatives to Prototyping

Alternatives to prototyping include simulation, formal analytical models, and architecture analysis. Simulation is viable primarily in mature domains, in which critical system properties can be abstracted and modeled.

Typically, formal analytical models are used in domains with critical requirements, such as hard real-time

deadlines or safety-critical properties. But the cost, steep learning curve, and difficulty of creating and scaling such models make them prohibitively expensive for most commercial projects.

Architecture analysis has been a relatively inexpensive way to gain early insight into a system's properties and risks. For traditional small-

performance or assess runtime resource consumption thresholds, scalability limits, and so on. Prototyping and architecture analysis each have pros and cons. So, we combine them.

We consider three kinds of prototypes. *Throwaway prototypes* are also called rapid prototypes or proofs of concept. In *vertical evolutionary prototypes*, one or more

Cost, schedule, and quality risks are more pronounced in big data system development.

data systems, architecture design is straightforward—it's an *N*-tier client-server design. So, architecture analysis has been largely taken for granted and omitted. However, in big data systems, many paradigm shifts underscore the importance of an explicit architecture design and analysis process.² Architecture analysis—understanding the consequences of design decisions—is critical and should occur early in the software lifecycle. It identifies not only architecture-related risks (for example, regarding performance, availability, security, modifiability, and integrability) but also process risks (for example, regarding requirements, tools, and methods) and organizational risks (for example, regarding the big picture, awareness, scope, and unrecognized needs).³

Our Approach

Architecture analysis usually takes much less time and costs much less than prototyping. However, it can't address all risks. For instance, only prototyping can verify a new big data component's real-time

system components are developed with full functionality in each release. Vertical evolutionary prototypes are generally more suitable for big data system development than horizontal evolutionary prototypes, as our case studies showed. A *minimum viable product* (MVP) is an evolutionary prototype that has only those core features that allow product deployment, minimizing the time spent on an iteration. An MVP emphasizes hypothesis testing by fielding the product with real users and collecting usage data. The data helps confirm or reject the hypothesis, letting developers make additional design decisions.

Cost (money, time, and effort) and effectiveness issues drove us to advocate strategic prototyping, but only in areas that architecture analysis can't adequately address. By strategic prototyping, we mean first employing an architecture-based approach to identify and analyze risks and then using less costly throwaway prototypes and vertical prototypes when necessary. This contrasts with building full (horizontal) evolutionary prototypes

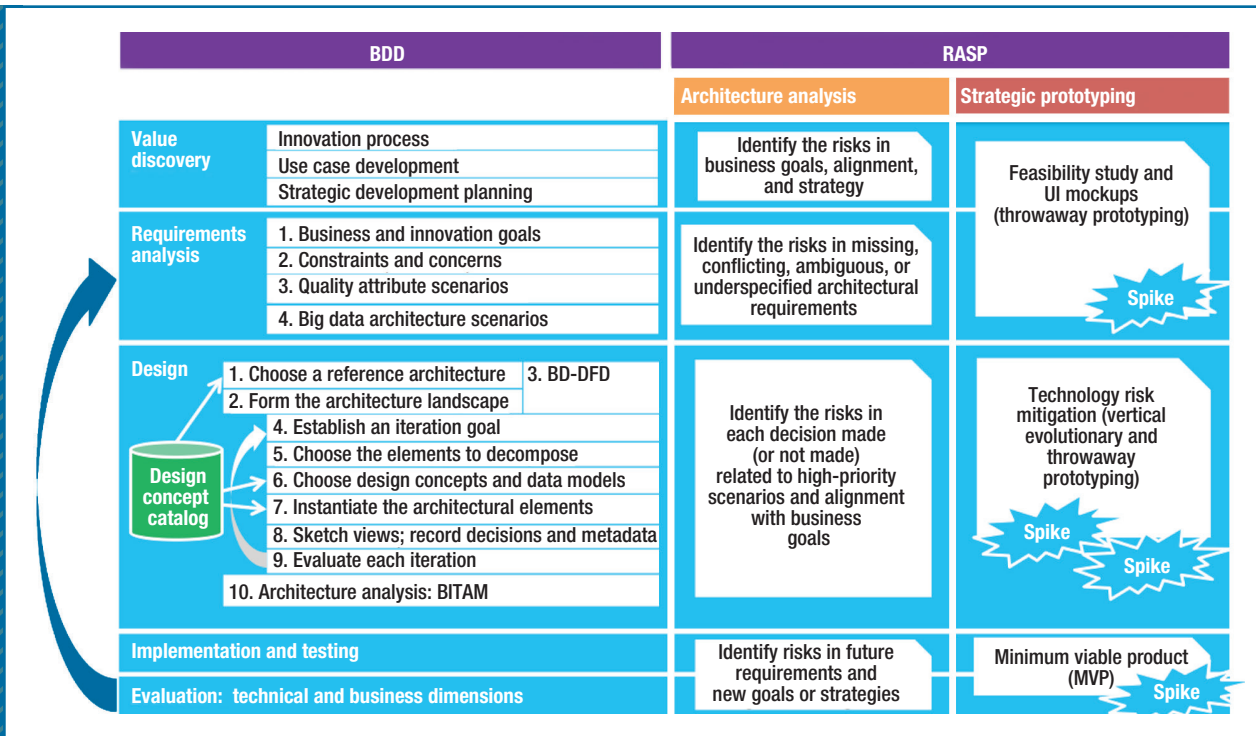


FIGURE 1. The RASP (Risk-Based Architecture-Centric Strategic Prototyping) model integrated with the BDD (Big Data Design) development methodology. BD-DFD stands for big data dataflow diagram; BITAM stands for Business and IT Alignment Method. A spike is a miniature project aimed at answering a specific quality attribute question.

blindly, as in conventional small-data system development.

The Case Studies

The nine case studies that we conducted at SoftServe covered

- network security and intrusion prevention,
- online-coupon Web analytics,
- a cloud-based mobile-app development platform,
- a telecom e-tailing platform,
- Web analytics and marketing optimization,
- healthcare insurance operational intelligence,
- an ultra-large-scale travel site,
- an operations intelligence platform for a content delivery network, and

- a big-data-as-a-service platform.

Details on the case studies are at <http://itm-vm.shidler.hawaii.edu/BDCases/CaseStudies.htm>.

The case studies and SoftServe's extensive practical experience show that big data system development differs qualitatively from small-data system development. In the case studies, most of the schedule and effort went into value discovery (from big data), architecture analysis, prototyping, and orchestration of off-the-shelf commercial and open source components. In contrast, in small-data system development, a far higher percentage of schedule and effort goes straight into programming. Given that the development processes differ, we need different

methods for designing and analyzing big data systems. And given that these systems are large, complex, and risky, an important aspect of this development process is determining whether, when, and where to prototype, as we'll show.

An Architecture-Centric Approach

RASP can stand alone, but we integrated it into BDD (Big Data Design), our architecture-centric agile development methodology (see Figure 1).^{1,2} BDD aims to address big data's inherent complexity (size, scale, heterogeneity, ambiguity, and so on). As Herbert Simon explained, architecture is the key to dealing with complexity: complex systems that exhibit near-decomposability

are easier to describe and understand and have simplified behavior.⁴ Because big data development focuses primarily on orchestration instead of coding, an architectural approach to technology selection and system integration is critical.

The relationship between architecture design and agility has been debated over the past decade. An architecture-centric approach facilitates rapid iterations with less disruption, as well as lower risk and cost, by creating appropriate architecture abstractions.⁵ For example, assume an architect anticipates that a technology family will rapidly change (by introducing capabilities or adding new members). If that architect is prudent, he or she will insert an abstraction layer as an intermediary between the technology family and the rest of the system, to insulate the system from such changes. So, an architecture-based mind-set can lay the foundation to support agility in both initial system construction and future iterations in big data system development.

The left side of Figure 1 outlines BDD, which differs from typical small-data system development.⁶ The latter typically comprises requirements analysis, conceptual design, selection of a database management system, logical design, physical design, prototyping, implementation, and performance evaluation. In contrast, BDD comprises value discovery, requirements analysis, design, implementation and testing, and evaluation of the technical and business dimensions.

Value Discovery

Our multiple-case study of 25 large enterprises revealed that the participants viewed big data as high risk and disruptive.⁷ So, all the

enterprises started development with value discovery. This phase encompasses an innovation process, use case development, and strategic deployment planning, including cost-benefit analysis,⁸ sourcing decisions, and talent management. The innovation process can be top-down, bottom-up, or a mix. It's often long, taking up to three years for some companies. High-level primary use cases that demonstrate the value of big data and opportunities for innovation are solicited or developed before any detailed requirements analysis. If companies don't find use cases that demonstrate value from big data, they typically don't continue with experimentation and deployment planning.

During value discovery, architecture analysis based on a rough, early design can help estimate the total cost of ownership and identify risks. This analysis includes eliciting key functional and quality attribute requirements as scenarios. Each scenario exercises some architectural alternatives; each alternative represents a significant set of architectural decisions. The risks

spikes to further understand technologies, clarify business and innovation goals and requirements, and demonstrate value. In agile methodologies, a spike is a miniature project, typically driven by a user story, aimed at answering a specific question. An architecture spike is similar but is driven by a quality attribute question, usually regarding performance; however, it could, in theory, address any system quality.⁹ In an architecture spike, developers typically create a prototype to address the quality attribute question. They could develop the spike on the main branch, but they typically develop it on a separate branch and (if it's successful) merge it with the main branch.

Requirements Analysis

Here, the regular agile lifecycle starts. The business and innovation goals as well as the design constraints and architectural concerns from requirements analysis will become inputs for system design. Analysis of the architectural requirements, expressed as quality attribute scenarios, helps identify additional risks: missing, conflicting, ambigu-

Architecture is the key to dealing with complexity.

determined in this phase focus on business strategy, the business goals' validity, and business-IT misalignment. This phase often involves strategic decisions on whether to develop an MVP.

Also during this phase, developers might then create low-to-medium-fidelity UI mockups or architecture

ous, or underspecified architecture requirements. The quality attributes (such as performance, security, modifiability, and availability), not functionality, drive architectural decisions. So, the requirements analysis activity must elicit, validate, and prioritize the quality attribute requirements because this will guide

subsequent architecture design and technology selection.

Design

This phase performs architecture design, data modeling, and technology selection in parallel, following the Attribute-Driven Design (ADD) method.^{1,2,10,11} An iterative process reuses well-known, assessed, and documented design primitives from a *design concepts catalog*² (an example catalog is at <http://smartdecisionsgame.com>). This makes architecture design simpler, more capable, and more repeatable. The design steps use three catalog components:

- reference architectures and frameworks;
- design patterns, tactics, and data models; and
- a big data technology catalog.

In step 1, architects choose a reference architecture for a data analytics application. For example, they could consult the design concepts catalog and see four options: extended relational, pure nonrelational, data refinery, and Lambda Architecture. They could also consult the catalog for specific technologies. For example, if they need a messaging technology for component integration, they could choose Apache Kafka, Amazon Simple Queue Service, Apache ActiveMQ, or RabbitMQ.

In step 2, architects create an architecture landscape; in step 3, they design big data dataflow diagrams.^{1,12} In steps 4 through 9, they decompose a system and a context diagram and model them in detail, in multiple iterations. Each iteration establishes goals (step 4) and selects a quality attribute, architectural concern, or system component to focus on (step 5). In step 6, architects select

the design concepts (such as patterns, tactics, technologies, and frameworks) and data models (based on inputs captured in the big data scenarios). In step 7, the instantiation of architectural elements includes selection of databases, analytics platforms, and other big data components. The technologies chosen for realizing conceptual components are critical architectural decisions; sometimes, steps 6 and 7 require experimentation—for example, a spike.

In step 8, the architects document the design decisions and their rationale. Step 9 evaluates the design decisions on the basis of iteration goals and measurements taken from any internal prototype or MVP.

In step 10, BDD employs BI-TAM (Business and IT Alignment Method),¹³ a lightweight architecture analysis method based on the Architecture Tradeoff Analysis Method,¹⁴ to identify risks and, in particular, misalignments. This analysis identifies risks concerning the architectural decisions made (or not made) relative to the high-priority scenarios. The risks aren't fixed at this point; risk mitigation is a separate, follow-on activity. Architects might mitigate the risks by

- changing the system's design, requirements, or business goals;
- making additional project management decisions (for example, retraining personnel or outsourcing parts of development); or
- prototyping, experiments, more-detailed analyses, or simulations.

Here, we focus on prototyping.

Implementation and Testing

In the implementation-and-testing phase, the architects flesh out the

architecture design and either replace prototypes (if they're throwaway) or evolve them (if they're evolutionary). And the architects plan and execute testing to confirm adherence to both the functional requirements and quality attribute requirements.

Evaluation

After a set of design iterations, the architects might choose to more completely analyze the architecture to discover risks and tradeoffs before committing to a release. At the end of each major development cycle or system release, architects can also perform architecture analysis to identify the risks of future requirements and new business goals and strategies. In addition, they can perform value discovery to continuously explore the anticipated value of the big data system they're constructing.

Applying RASP

Because prototyping decisions are critical in big data system development, architects should make them in a consistent, disciplined way. Because technology orchestration is the core of big data system development, such development relies heavily on prototyping and architecture analysis for risk management. As we've shown, RASP can guide this prototyping and analysis.

On the basis of the lessons learned from our research with SoftServe, we developed questions that architects should answer and a decision flowchart (see Figure 2) for them to follow before they embark on a (relatively costly) prototyping effort.

In our case studies, employing standardized questions and decision procedures as well as architecture analysis methods helped us plan

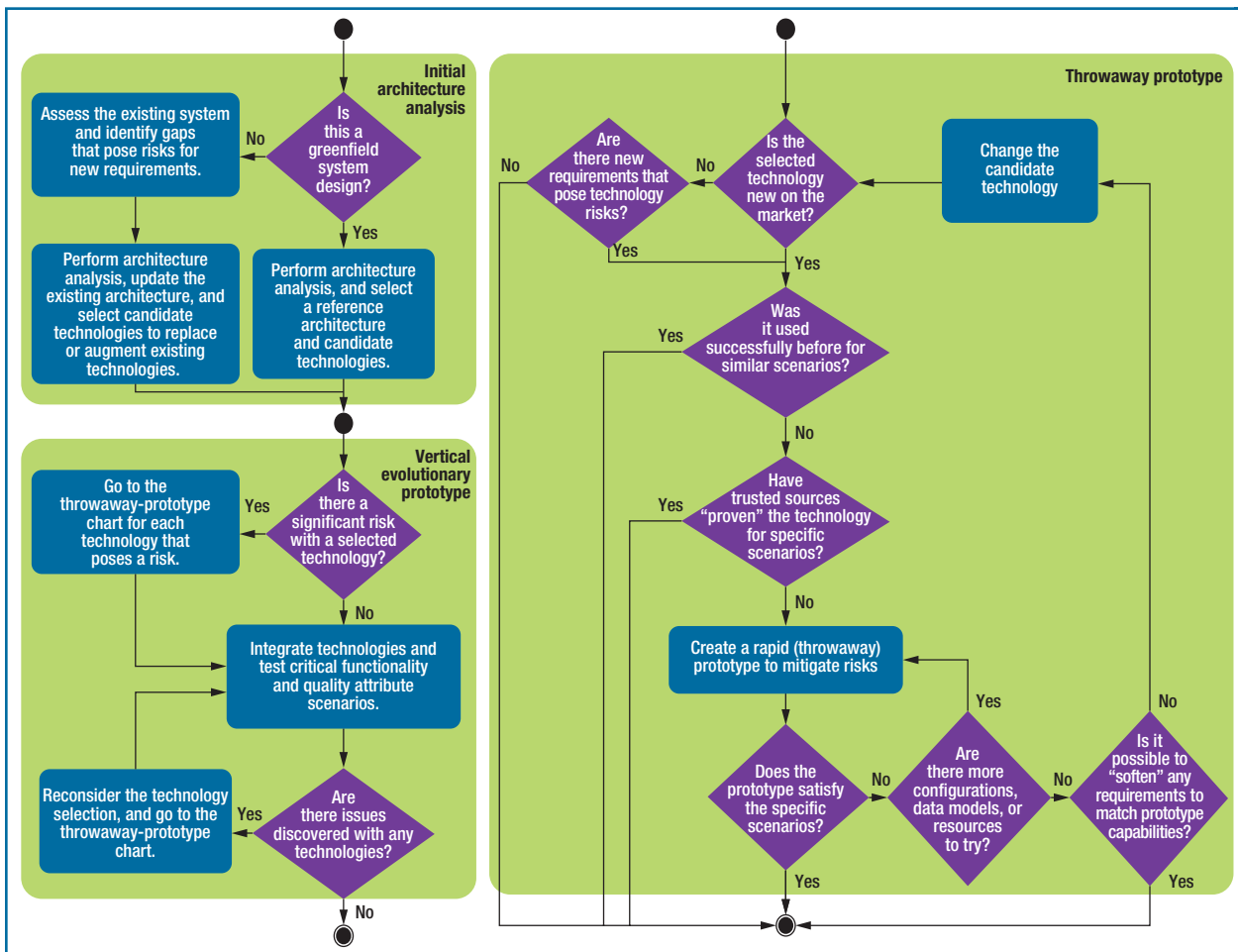


FIGURE 2. A strategic-prototyping decision flowchart. Employing it will help software architects make decisions in a consistent, disciplined way.

prototyping systematically and strategically. For example, the questions focused on whether

- the technology was new,
- we had successfully used it before,
- significant new requirements existed, and
- objective evidence substantiated product performance claims.

The answers to these questions helped us determine whether to prototype and how to evaluate the prototyping

results. For example, an unsuccessful prototype might result in choosing a different technology, but another option is to soften some requirements. Our case studies exemplified each of these kinds of decisions.

In each case study, we applied RASP alongside BDD (and its design concepts catalog) to systematically design and manage risk. Because each project had different business and innovation goals, their risks differed. However, consistent risk themes emerged: technology selection, performance of new

technologies, total cost of ownership, and integration of technologies.

We didn't create a prototype unless we had determined that architecture analysis alone was inadequate to address risk. For two of the case studies, which required quick feedback on the budget and schedule for business decision making, architecture analysis alone sufficed. The other seven case studies strategically employed prototyping, which helped minimize risk and was cost-effective. In each of these seven case studies, creating vertical evolutionary

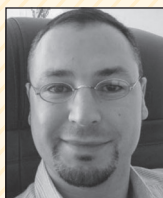
ABOUT THE AUTHORS



HONG-MEI CHEN is a professor of IT management in the Shidler College of Business at the University of Hawaii at Manoa. Her research interests include novel information system design and development methodologies, big data engineering and management, social customer relationship management, service engineering, software architecture, and business-IT alignment. Chen received a PhD in business administration from the University of Arizona. Contact her at hmchen@hawaii.edu.



RICK KAZMAN is a professor of information technology management at the University of Hawaii at Manoa and a principal researcher at the Software Engineering Institute. His research interests include software architecture, design and analysis tools, software visualization, and software engineering economics. Kazman received a PhD in computational linguistics from Carnegie Mellon University. He's a senior member of the IEEE Computer Society. Contact him at kazman@hawaii.edu.



SERGE HAZIYEV is a vice president of software architecture at SoftServe. He leads the Architecture Group and designs, evaluates, and modernizes large-scale software solutions. He has a Software Engineering Institute Software Architecture Professional certificate and more than 17 years' experience in enterprise-level solutions including big data, software as a service and cloud computing, service-oriented architecture, and carrier-grade telecommunication services. Contact him at shaziyeu@softserveinc.com.

prototyping. Analysis helps you select candidate technologies; prototyping validates those choices and quickly gets a working system in front of the customer. This aids in eliciting feedback and prioritizing future development efforts.

- An evolutionary prototype can effectively mitigate risk if you implement it as a skeleton—an infrastructure into which you can integrate components and technologies.¹⁴ This helps mitigate integration risks, supports early demonstrations of user value, and aids the understanding of end-to-end scenarios.
- Vertical evolutionary prototyping can help answer questions about systemwide properties but might need to be augmented with throwaway prototypes when requirements are volatile. When requirements change, new architecture analysis and throwaway prototypes might be needed (to help avoid falling into the “hammer looking for a nail” syndrome).
- Throwaway prototypes work best when you need to quickly evaluate a technology, to see whether it satisfies critical architecture scenarios (usually related to quality attributes such as performance and scalability). They're typically used for narrow scenarios that involve evaluating just one or two components.
- Whether to create an MVP is more of a business decision than a decision driven simply by technological risk. Typical business goals addressed include getting early user feedback and updating the product accordingly, presenting a working version to a trade show or customer event,

prototypes and throwaway prototypes took from two to six weeks, much less time than for horizontal evolutionary prototyping (although creating and evolving an MVP might take longer).

On the basis of these case studies and our prior experience, we created and verified seven guidelines for when to do architecture analysis, when to prototype, and under what conditions to deploy each kind of prototype:

- Architecture analysis might be the only feasible option when you need to make early decisions that have a project-wide scope. Then, you can employ throwaway prototypes to make technology choices and demonstrate feasibility.
- Architecture analysis alone is insufficient to prove many important system properties.
- Architecture analysis complements vertical evolutionary

structuring the development process, checking team progress and alignment, or testing an outsourcing vendor using an MVP as a pilot project.

Richard Baskerville and Jan Stage advocated risk analysis for controlling evolutionary prototyping;¹⁵ we've expanded on this position. Our research, departing from prior prototyping approaches, demonstrated that an architecture-centric approach—RASP—can help make risk management explicit, systematic, and cost-effective. The need for strategic prototyping is critical for big data system development, which involves high risk, high complexity, and immature technologies. RASP's implementation of strategic prototyping provides a basis for value discovery with stakeholders, for reasoning about risks, for planning and estimating cost and schedule, and for supporting experimentation.

Prior studies analyzed how organizations decide whether to use prototyping and what environmental and organizational variables (such as project size and developer, user, and organizational characteristics) affect the proper use of prototyping.¹⁶ However, these studies produced no significant findings. Other researchers created a risk list to guide risk management, but that list involved more than 100 factors, rendering it impractical.³ We believe our architecture-centric approach, based on primary use cases and prioritized quality attribute scenarios, provides a more intuitive, efficient, and effective way to identify, mitigate, and continuously monitor risks.

In particular, our strategic prototyping decision flowchart and

the associated guidelines help make RASP repeatable and prototyping cost-effective. Architects can plan and manage prototyping in a strategic, disciplined fashion. However, no design or development method, no matter how thorough, can guarantee success. The architectural agility created and the architecture analyses performed with RASP rely on the architect's training and discipline. 

References

1. H.-M. Chen, R. Kazman, and S. Haziyevev, "Agile Big Data Analytics Development: An Architecture-Centric Approach," *Proc. 2016 Hawaii Int'l Conf. System Sciences (HICSS 16)*, 2016, pp. 5378–5387.
2. H.-M. Chen et al., "Big Data System Development: An Embedded Case Study with a Global Outsourcing Firm," *Proc. 1st Int'l Workshop Big Data Software Eng. (BIGDSE 15)*, 2015, pp. 44–50.
3. L. Bass et al., *Risk Themes Discovered through Architecture Evaluations*, tech. report CMU/SEI-2006-TR-012, Software Eng. Inst., Carnegie Mellon Univ., 2006.
4. H. Simon, "The Architecture of Complexity," *Proc. Am. Philosophical Soc.*, vol. 106, no. 6, 1962, pp. 467–482.
5. S. Bellomo, I. Gorton, and R. Kazman, "Toward Agile Architecture: Insights from 15 Years of ATAM Data," *IEEE Software*, vol. 32, no. 5, 2015, pp. 38–45.
6. R. Elmasri and S. Navathe, *Fundamentals of Database Systems*, 6th ed., Addison-Wesley, 2010.
7. H.-M. Chen et al., "Amazon in the Air: Innovating with Big Data at Luft-hansa," *Proc. 2016 Hawaii Int'l Conf. System Sciences (HICSS 16)*, 2016, pp. 5096–5105.
8. J. Asundi, R. Kazman, and M. Klein, "Using Economic Considerations to Choose among Architecture Design Alternatives," tech report CMU/SEI-2001-TR-035, Software Eng. Inst., Carnegie Mellon Univ., 2001.
9. T.C.N. Graham, R. Kazman, and C. Walmsley, "Agility and Experimentation: Practical Techniques for Resolving Architectural Tradeoffs," *Proc. 29th Int'l Conf. Software Eng. (ICSE 07)*, 2007, pp. 519–528.
10. H. Cervantes, P. Velasco-Elizondo, and R. Kazman, "A Principled Way to Use Frameworks in Architecture Design," *IEEE Software*, vol. 30, no. 2, 2013, pp. 46–53.
11. H. Cervantes and R. Kazman, *Designing Software Architectures: A Practical Approach*, Addison-Wesley, to be published in 2016.
12. H.-M. Chen and R. Kazman, "Architecting Ultra-Large-Scale Green Information Systems," *Proc. 1st Int'l Workshop Green and Sustainable Software (GREENS 12)*, 2012, pp. 69–75.
13. H.-M. Chen, R. Kazman, and A. Garg, "BITAM: An Engineering-Principled Method for Managing Misalignments between Business and IT Architectures," *Science of Computer Programming*, vol. 57, no. 1, 2005, pp. 5–26.
14. L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed., Addison-Wesley, 2012.
15. R.L. Baskerville and J. Stage, "Controlling Prototype Development through Risk Analysis," *MIS Q.*, vol. 20, no. 4, 1996, pp. 481–504.
16. B. Hardgrave, R. Wilson, and K. Eastman, "Toward a Contingency Model for Selecting an Information System Prototyping Strategy," *J. Management Information Systems*, vol. 15, no. 2, 1999, pp. 113–136.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.